

Part A: Variables and Data Types

Q1. Your first variable (declare and print)

basic form: type name = value;

Declare an integer variable named age and give it the value 20. Then print it using printf.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>
int main (void)
{
    int age = 20;
    printf ("%i", age);
}
```

Q2. One variable of each type

Declare four variables, one of each type, and print each on its own line: an int named count with value 5, a float named price with value 9.99, a char named grade with value 'A', and a string named student with value "Ali". Use the correct format specifier for each (%i, %f, %c, %s).

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>
int main(void)
{
    int count = 5;
    float price = 9.99;
    char grade = 'A';
    string student = "Ali";

    printf ("%i\n", count);
    printf ("%f\n", price);
    printf ("%c\n", grade);
    printf ("%s\n", student);
}
```

taking input from the user (CS50 style)

Ask the user for their name and their age, store them, and print: Hello <name>, you are <age> years old. Use `get_string` for the name and `get_int` for the age. Remember: the input function should match the data type.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>

int main (void)
{
    string name = get_string ("Enter your name");
    int age = get_int ("Enter your age");
    printf ("Hello %s, you are %i years old\n", name, age);
}
```

Part B: Operators

Q4. Arithmetic on two numbers

Input two integers a and b. Print their sum, difference, product, quotient, and remainder, each clearly labelled.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int a = get_int("Enter a:");
    int b = get_int("Enter b:");

    int sum = a + b;
    int sub = a - b;
    int mult = a * b;
    int divi = a / b; int rem = a % b;
```

```
    printf("sum: %d\n", sum);
    printf("difference: %d\n", sub);
    printf("product: %d\n", mult);
    printf("quotient: %d\n", divi);
    printf("remainder: %d\n", rem);
```

}

shorthand and tracing (no code, just reason)

updated form of a variable

Given `int x = 10;`, write the value of `x` after each line runs, in order:

```
x += 5;    // x is now 15
x -= 3;    // x is now 12
x *= 2;    // x is now 24
x++;       // x is now 25
```

This is the "updated" way of using a variable: declare it once, then keep changing it.

Write the four values and a one-line explanation for each:

15 $\rightarrow x = 10 + 5 \rightarrow x = 15$
12 $\rightarrow x = 15 - 3 \rightarrow x = 12$
24 $\rightarrow x = 12 * 2 \rightarrow x = 24$
25 $\rightarrow x = 24 + 1 \rightarrow x = 25$

Q6. Boolean expression

Write a single Boolean expression that is true when score is between 50 and 100 (inclusive). You do not need a full program, just the expression.

Write your Boolean expression here:

`x >= 50 && x <= 100`

Part C: Boolean Expressions and Conditions

Q7. Positive, negative, or zero

Input an integer. Using if / else if / else, print whether it is Positive, Negative, or Zero.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
```

```
#include <cs50.h>
```

```
int main (void)
```

```
{ int n;
```

```
  if (n < 0)
```

```
  { printf ("Negative %.d", n); }
```

```
  else if (n = 0n > 0)
```

```
  { printf ("Zero %.d", n); }
```

```
  else  
  { printf ("Positive %.d", n); }
```

```
}
```

8. Grade classification (else-if ladder)

Input marks (0 to 100). Print the grade: A if marks ≥ 80 , B if marks ≥ 60 and < 80 , C if marks < 60 .

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
```

```
#include <cs50.h>
```

```
int main (void)
```

```
{ int marks = get_int ("Marks: ");
```

```
    if (marks  $\geq$  80)
```

```
    { printf ("A\n"); }
```

```
    else if (marks  $\geq$  60)
```

```
    { printf ("B\n"); }
```

```
    else
```

```
    { printf ("C\n"); }
```

```
}
```

Q9. Nested if: ticket price

Input the person's age and student status (1 for student, 0 for not). Using a nested if, print the ticket price:

- If age < 12, price = 50
- If age is between 12 and 60 (inclusive): student → 80, not a student → 100
- If age > 60, price = 60

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>

int main (void)
{
    int age = get_int("Age:");
    int student = get_int("Student (1 for yes, 0 for no); ");

    if (age < 12)
    {
        printf("Price: 50\n");
    }
    else if (age <= 60)
    {
        if (student == 1)
        {
            printf("Price: 80\n");
        }
        else
        {
            printf("Price: 100\n");
        }
    }
    else
    {
        printf("Price: 60\n");
    }
}
```

Part D: Loops with Dry Runs

Q10. Count up (for loop)

Write a for loop that prints the numbers 1 to 10, each separated by a space.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>

int main (void)
{
    for (int i = 1; i <= 10; i++)
    {
        printf("%i ", i);
    }
}
```

Now fill in the dry run table (add rows if you need them):

Step	Loop variable (i)	Condition	Output so far
	1	1 <= 10 T	1
	2	2 <= 10 T	2
	3	3 <= 10 T	3
	4	4 <= 10 T	4
	5	5 <= 10 T	5
	6	6 <= 10 T	6
	7	7 <= 10 T	7
	11	11 <= 10 F	X

Q11. Count down (while loop)

Write a while loop that prints the numbers 5 down to 1.

Write your C code here (and type the same in CS50s.dev):

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int i = 5;
    while (i <= 5 && i > 0)
    {
        printf("%i\n", i);
        i--;
    }
}
```

Now fill in the dry run table:

Step	Loop variable (i)	Condition	Output so far
	5	T	5
	4	T	4
	3	T	3
	2	T	2
	1	T	1
	0	F	X
	6	F	X

2. Sum from 1 to n (accumulator variable)

Input a number n . Use a loop to add up $1 + 2 + \dots + n$ and print the total. Here you declare a sum variable, start it at 0, and update it each step. This is the accumulator pattern.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>
int main(void)
{
    int sum = 0;
    int n = get_int("Enter a number:");
    for (int i = 1; i <= n; i++)
    {
        sum = sum + i;
        printf("sum = %d\n", sum);
    }
}
```

Now fill in the dry run table: $n=5$

Step	Loop variable (i)	sum so far	Explanation
	1	$0+1=1$	$sum = sum + 1$ $sum = 0, i = 1$
	2	$1+2=3$	$sum = 1, i = 2$
	3	$3+3=6$	$sum = 3, i = 3$
	4	$4+6=10$	$sum = 6, i = 4$
	5	$5+10=15$	$sum = 10, i = 5$

Q3. Factorial (accumulator with multiplication)

The factorial of n is $1 \times 2 \times 3 \times \dots \times n$. Input n (assume $n \geq 0$), compute $n!$ with a loop, and print it. Example: input 5 → output 120.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>
int main(void)
{
    int fac = 1;
    int n = get_int("Enter a number: ");
    for (int i = 1; i <= n; i++)
    {
        fac = fac * i;
    }
    printf("%i", fac);
}
```

Now fill in the dry run table:

Step	Loop variable (i)	product so far	Explanation
	1	1	Fac = 1 i = 1 fac = fac * i Fac = 1 * 1
	2	2	Fac = 1 i = 2 1 * 2 = 2
	3	6	Fac = 2 i = 3 6
	4	24	Fac = 6 i = 4 24
	5	120	Fac = 24 i = 5 120

Part E: Arrays and Functions

Q14. Declare, fill, and print an array with a loop

Declare an integer array numbers of size 5, initialize it with the values (10, 20, 30, 40, 50), then use a for loop to print every element on its own line. Remember: the first index is 0 and the last index is size - 1.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>
int main(void)
{
    int main[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++)
    {
        printf("Num [%i] = %d\n", i, main[i]);
    }
}
```

write the prototype above main(), the definition below

Q18. Your first function

Write a user-defined function `int is_even(int n)` that returns true if `n` is even and false otherwise. In `main()`, input a number, call `is_even`, and print Even or Odd.

Write your C code here (and type the same in CS50.dev):

```
#include <stdio.h>
#include <cs50.h>

int main(void)
{
    int number = get_int("Number: ");
    if (is_even(number) == 1)
    {
        printf("Even\n");
    }
    else
    {
        printf("Odd\n");
    }
}

if (n % 2 == 0)
{
    return 1;
}
```

```
else
{
    return 0;
}
```

}

— End of Worksheet —

CSCI-1001: Introduction to Computing & Programming

Worksheet: Learning to Code in C, Step by Step

Asian University for Women (AUW)

Name:

Mo Hnang Sing Maome

Student ID:

242215

Section:

04

Date:

10/02/26

How to use this worksheet

- Work through the questions in order. Each one builds on the one before it.
- For every question, write the complete C code by hand in the box provided, then type and run the same code in CS50.dev.
- For every loop question, you must also fill in the dry run table.
- Assume `stdio.h` and `cs50.h` are already included.
- Handwritten code, dry run, and submitted .c file must match. Copied or hardcoded answers receive zero.

How to write a dry run (read this first)

A dry run means you become the computer. You go through the loop one step at a time and write down what happens, *without* running the program. This is how you learn to predict output instead of guessing.

At each step you track three things: the loop variable, whether the condition is true or false, and what gets printed.

Worked example. Trace this loop:

```
int i;
for (i = 1; i <= 3; i++)
{
    printf("%d ", i);
}
```

Step	Loop variable (i)	Condition (i <= 3)	Output so far
1	i = 1	1 <= 3 → true	1
2	i = 2	2 <= 3 → true	1 2
3	i = 3	3 <= 3 → true	1 2 3
4	i = 4	4 <= 3 → false	loop ends

The loop stops the moment the condition is false. Final output: 1 2 3. You will fill in tables like this for every loop question below.